L31 — Breadth First Search (BFS)

Unit: 3 | **Chapter:** Graphs | **BT Level:** BT5 | **CO:** CO2, CO3

Learning Objectives

- Explain the BFS strategy and its level-by-level exploration
- Implement BFS using a queue and visited set
- Trace BFS on both directed and undirected graphs
- Use BFS to find shortest paths in unweighted graphs

---

1. BFS Concept

Breadth-First Search (BFS) explores a graph level by level, visiting all vertices at distance k before visiting any vertex at distance $k+1$.

Intuition: Like ripples spreading outward from a stone dropped in water.

```
Graph:            BFS from A:
  A               Level 0: A
 / \              Level 1: B, C
B   C             Level 2: D, E, F
|  / \
D E   F
```

Key properties:

- Uses a **queue** (FIFO) to manage exploration order
- Marks vertices as **visited** to avoid reprocessing
- Finds **shortest path** (in terms of number of edges) in unweighted graphs

---

2. BFS Algorithm

```
Algorithm BFS(graph, source):
  visited = {source}
  queue = Queue()
  queue.enqueue(source)
  parent = {source: None}

  while queue is not empty:
      u = queue.dequeue()
      process(u)   // visit / record
      for each neighbor v of u:
          if v not in visited:
              visited.add(v)
              parent[v] = u
              queue.enqueue(v)
```

```
  return parent   // BFS tree / shortest paths
```

---

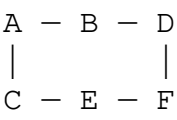
3. Implementation

```
from collections import deque, defaultdict

def bfs(graph, source):
    """
    BFS from source vertex.
    Returns: (visited order, parent map, distance from source)
    """
    visited = {source}
    queue = deque([source])
    parent = {source: None}
    distance = {source: 0}
    order = []

    while queue:
        u = queue.popleft()
        order.append(u)

        for v in graph.get(u, []):
            if v not in visited:
                visited.add(v)
                parent[v] = u
                distance[v] = distance[u] + 1
                queue.append(v)

    return order, parent, distance


def shortest_path(graph, source, target):
    """Find shortest path from source to target using BFS."""
    _, parent, distance = bfs(graph, source)

    if target not in parent:
        return None, float('inf')   # Not reachable

    # Reconstruct path
    path = []
    node = target
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()

    return path, distance[target]
```

---

4. Detailed Trace

Graph:



Edges: A-B, A-C, B-D, C-E, D-F, E-F

BFS from A:

Step	Dequeue	Queue after	Visited	Action
1	—	[A]	{A}	Enqueue source A
2	A	[B, C]	{A,B,C}	Process A, enqueue B,C
3	B	[C, D]	{A,B,C,D}	Process B, enqueue D
4	C	[D, E]	{A,B,C,D,E}	Process C, enqueue E
5	D	[E, F]	{A,B,C,D,E,F}	Process D, enqueue F
6	E	[F]	{A,B,C,D,E,F}	Process E, F already visited
7	F	[]	{A,B,C,D,E,F}	Process F (all neighbors visited)

BFS order: A → B → C → D → E → F
Distances from A: A = 0, B = 1, C = 1, D = 2, E = 2, F = 3
Shortest path A→F: A → B → D → F (length 3) or A → C → E → F (length 3)

---

5. BFS on Disconnected Graphs

```
def bfs_all_components(graph):
    """Handle disconnected graphs — run BFS from each unvisited vertex.
    all_visited = set()
    components = []

    for vertex in graph:
        if vertex not in all_visited:
            order, parent, distance = bfs(graph, vertex)
            components.append(order)
            all_visited.update(order)

    return components
```

---

6. Applications

6.1 Shortest Path (Unweighted)

BFS gives the minimum number of edges between source and any reachable vertex.

6.2 Level-Order Tree Traversal

```
def level_order_traversal(tree, root):
    """BFS on a tree = level-order traversal."""
```

```
    _, _, distance = bfs(tree, root)
    levels = defaultdict(list)
    for node, d in distance.items():
        levels[d].append(node)
    return [levels[k] for k in sorted(levels)]
```

6.3 Bipartite Graph Check

```
def is_bipartite(graph):
    """
    A graph is bipartite iff it has no odd-length cycles.
    Use BFS 2-coloring to check.
    """
    color = {}

    for start in graph:
        if start in color:
            continue
        color[start] = 0
        queue = deque([start])

        while queue:
            u = queue.popleft()
            for v in graph.get(u, []):
                if v not in color:
                    color[v] = 1 - color[u]   # Alternate color
                    queue.append(v)
                elif color[v] == color[u]:
                    return False   # Same color → odd cycle → not bipar

    return True
```

6.4 Finding All Nodes at Distance K

```
def nodes_at_distance_k(graph, source, k):
    """Return all nodes exactly k hops from source."""
    _, _, distance = bfs(graph, source)
    return [v for v, d in distance.items() if d == k]
```

---

7. Complexity Analysis

Graph Representation	Time	Space
Adjacency List	$O(V + E)$	$O(V)$
Adjacency Matrix	$O(V^2)$	$O(V)$

- Every vertex is enqueued/dequeued at most once: $O(V)$
- Every edge is inspected at most twice (undirected): $O(E)$
- Total: **$O(V + E)$** with adjacency list

Summary

- BFS explores vertices level by level using a **queue**.
- Marks vertices visited **before** enqueueing (not after) to avoid duplicate entries.
- Guarantees **shortest path** in unweighted graphs.
- Time: O(V + E) with adjacency list representation.
- Applications: shortest path, level-order traversal, bipartite check, connected components.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 22.2 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 5.6 (Springer, 2020)